Online Retail Transaction Access and Returns Analytics

SQL Phase 2 Capstone Project - Prepared for Shraddha Yadav

1. Domain, Dataset and End Consumer

Domain: Employment & Poverty / Consumer Access - online retail affordability, market reach and returns

Dataset: online_retail.csv

Cleaned SQLite table name: online_retail_transactions

Rows used in project database: 5,000. Columns: 15.

End consumer: A retail operations and customer-access analyst who needs to understand affordability, product performance, customer reach, return risk, and operational planning issues from transaction/product data.

Important approval note: this pack is approval-first. Send the dataset approval email before treating the SQL/report as final submission.

2. Five Analytical Perspectives

1. Revenue and Market Coverage
2. Customer and Access Analysis
3. Product Demand and Catalogue Analysis
4. Returns, Cancellations and Data Quality
5. Time Trends and Operational Planning

3. SQL Concept Coverage

Concept	Number of queries
SELECT	7
WHERE	18
GROUP BY	25
CTE	9
VIEW	5
Window Function	11

4. Query Outputs and Interpretations

Q01. What are the first 10 transaction records?

Perspective: Revenue and Market Coverage | Difficulty: Easy | SQL concept: SELECT

SQL used:

```
SELECT invoice_no, stock_code, description, quantity, unit_price, country FROM online_retail_transactions LIMIT 10;
```

Output sample:

invoice_no	stock_code	description	quantity	unit_price	country
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2.55	United Kingdom
536381	22411	JUMBO SHOPPER	10	1.95	United Kingdom

		VINTAGE RED PAISLEY			
536390	22968	ROSE COTTAGE KEEPSAKE BOX	8	8.5	United Kingdom
536401	85049E	SCANDINAVIAN REDS RIBBONS	2	1.25	United Kingdom
536407	22632	HAND WARMER RED POLKA DOT	6	1.85	United Kingdom

Interpretation: Shows a quick sample of the transaction data.

Q02. Which transactions are from the United Kingdom?

Perspective: Revenue and Market Coverage | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, description, quantity, unit_price, country FROM online_retail_transactions WHERE country = 'United Kingdom'
LIMIT 20;
```

Output sample:

invoice_no	description	quantity	unit_price	country
536365	WHITE HANGING HEART T-LIGHT HOLDER	6	2.55	United Kingdom
536381	JUMBO SHOPPER VINTAGE RED PAISLEY	10	1.95	United Kingdom
536390	ROSE COTTAGE KEEPSAKE BOX	8	8.5	United Kingdom
536401	SCANDINAVIAN REDS RIBBONS	2	1.25	United Kingdom
536407	HAND WARMER RED POLKA DOT	6	1.85	United Kingdom

Interpretation: Filters the dataset to UK transactions.

Q03. Which lines have a positive revenue value?

Perspective: Revenue and Market Coverage | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, description, line_revenue FROM online_retail_transactions WHERE line_revenue > 0 ORDER BY line_revenue DESC
LIMIT 20;
```

Output sample:

invoice_no	description	line_revenue
550461	GIN + TONIC DIET METAL SIGN	3380.0
548813	Manual	2382.92
572869	RABBIT NIGHT LIGHT	1718.4
547708	CAST IRON HOOK GARDEN FORK	871.56
551191	PAPER CHAIN KIT EMPIRE	816.0

Interpretation: Finds high-value paid transaction lines.

Q04. Which countries appear in the dataset?

Perspective: Revenue and Market Coverage | Difficulty: Easy | SQL concept: SELECT

SQL used:

```
SELECT DISTINCT country FROM online_retail_transactions WHERE country <> '' ORDER BY country LIMIT 50;
```

Output sample:

country
Australia

Austria
Bahrain
Belgium
Canada

Interpretation: Lists different customer countries.

Q05. Which transactions have unit prices above 20?

Perspective: Revenue and Market Coverage | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, description, unit_price, country FROM online_retail_transactions WHERE unit_price > 20 ORDER BY unit_price
DESC LIMIT 20;
```

Output sample:

invoice_no	description	unit_price	country
548813	Manual	2382.92	Singapore
541421	DOTCOM POSTAGE	449.52	United Kingdom
551009	DOTCOM POSTAGE	264.0	United Kingdom
542115	DOTCOM POSTAGE	247.54	United Kingdom
563093	DOTCOM POSTAGE	176.41	United Kingdom

Interpretation: Shows higher-priced items that may drive revenue.

Q06. Which countries generate the highest total revenue?

Perspective: Revenue and Market Coverage | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, ROUND(SUM(line_revenue),2) AS total_revenue, COUNT(*) AS transaction_lines FROM online_retail_transactions
GROUP BY country ORDER BY total_revenue DESC LIMIT 15;
```

Output sample:

country	total_revenue	transaction_lines
United Kingdom	78969.56	4588
EIRE	3406.78	68
Singapore	2622.02	4
Netherlands	2153.3	22
Japan	2043.48	5

Interpretation: Groups by country to compare revenue contribution.

Q07. What is the average order line value by country?

Perspective: Revenue and Market Coverage | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, ROUND(AVG(line_revenue),2) AS avg_line_revenue, COUNT(*) AS lines FROM online_retail_transactions WHERE
line_revenue > 0 GROUP BY country HAVING COUNT(*) >= 5 ORDER BY avg_line_revenue DESC LIMIT 15;
```

Output sample:

country	avg_line_revenue	lines
Japan	408.7	5
Netherlands	97.88	22
Australia	87.2	10
EIRE	51.86	66
Channel Islands	34.03	10

Interpretation: Compares typical transaction values across countries.

Q08. Which countries have the most unique customers?

Perspective: Revenue and Market Coverage | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, COUNT(DISTINCT customer_id) AS unique_customers FROM online_retail_transactions WHERE customer_id IS NOT NULL
GROUP BY country ORDER BY unique_customers DESC LIMIT 15;
```

Output sample:

country	unique_customers
United Kingdom	1666
Germany	49
France	33
Belgium	13
Switzerland	12

Interpretation: Shows customer reach by country.

Q09. Which countries have the highest total quantity sold?

Perspective: Revenue and Market Coverage | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, SUM(quantity) AS total_quantity FROM online_retail_transactions WHERE quantity > 0 GROUP BY country ORDER BY
total_quantity DESC LIMIT 15;
```

Output sample:

country	total_quantity
United Kingdom	43789
EIRE	1924
Japan	1583
Netherlands	1423
Germany	1121

Interpretation: Identifies markets with high sales volume.

Q10. Which countries have unusually low customer coverage?

Perspective: Revenue and Market Coverage | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, COUNT(DISTINCT customer_id) AS unique_customers, COUNT(*) AS lines FROM online_retail_transactions GROUP BY
country HAVING unique_customers < 10 ORDER BY lines DESC LIMIT 20;
```

Output sample:

country	unique_customers	lines
EIRE	3	68
Netherlands	2	22
Spain	9	19
Norway	6	10
Channel Islands	6	10

Interpretation: Flags countries with fewer identified customers.

Q11. Which top countries contribute most to overall revenue share?

Perspective: Revenue and Market Coverage | Difficulty: Difficult | SQL concept: CTE

SQL used:

```
WITH country_revenue AS (SELECT country, SUM(line_revenue) AS revenue FROM online_retail_transactions WHERE line_revenue > 0
GROUP BY country), total_revenue AS (SELECT SUM(revenue) AS total_rev FROM country_revenue) SELECT country, ROUND(revenue,2)
```

```
AS revenue, ROUND(100.0*revenue/total_rev,2) AS pct_of_total FROM country_revenue, total_revenue ORDER BY revenue DESC LIMIT 15;
```

Output sample:

country	revenue	pct_of_total
United Kingdom	80225.96	81.33
EIRE	3422.48	3.47
Singapore	2622.02	2.66
Netherlands	2153.3	2.18
Japan	2043.48	2.07

Interpretation: Uses CTEs to calculate country revenue and percentage share.

Q12. How do countries rank by revenue?

Perspective: Revenue and Market Coverage | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
SELECT country, ROUND(SUM(line_revenue),2) AS total_revenue, RANK() OVER (ORDER BY SUM(line_revenue) DESC) AS revenue_rank
FROM online_retail_transactions WHERE line_revenue > 0 GROUP BY country ORDER BY revenue_rank LIMIT 15;
```

Output sample:

country	total_revenue	revenue_rank
United Kingdom	80225.96	1
EIRE	3422.48	2
Singapore	2622.02	3
Netherlands	2153.3	4
Japan	2043.48	5

Interpretation: Ranks countries by revenue using a window function.

Q13. What is each country revenue compared with the previous rank?

Perspective: Revenue and Market Coverage | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
WITH ranked AS (SELECT country, SUM(line_revenue) AS revenue, RANK() OVER (ORDER BY SUM(line_revenue) DESC) AS rnk FROM
online_retail_transactions WHERE line_revenue > 0 GROUP BY country) SELECT country, ROUND(revenue,2) AS revenue, rnk,
ROUND(revenue - LAG(revenue) OVER (ORDER BY rnk),2) AS gap_from_previous FROM ranked ORDER BY rnk LIMIT 15;
```

Output sample:

country	revenue	rnk	gap_from_previous
United Kingdom	80225.96	1	None
EIRE	3422.48	2	-76803.48
Singapore	2622.02	3	-800.46
Netherlands	2153.3	4	-468.72
Japan	2043.48	5	-109.82

Interpretation: Uses LAG to compare country revenue gaps.

Q14. Create a reusable country revenue view and show the top countries.

Perspective: Revenue and Market Coverage | Difficulty: Difficult | SQL concept: VIEW

SQL used:

```
SELECT country, ROUND(total_revenue,2) AS total_revenue, transaction_lines FROM v_country_revenue ORDER BY total_revenue DESC
LIMIT 15;
```

Output sample:

country	total_revenue	transaction_lines
United Kingdom	80225.96	4494

EIRE	3422.48	66
Singapore	2622.02	4
Netherlands	2153.3	22
Japan	2043.48	5

Interpretation: Uses a reusable view for country revenue analysis.

Q15. Which countries are high revenue but have fewer customers?

Perspective: Revenue and Market Coverage | Difficulty: Difficult | SQL concept: CTE

SQL used:

```
WITH country_metrics AS (SELECT country, SUM(line_revenue) AS revenue, COUNT(DISTINCT customer_id) AS customers FROM online_retail_transactions WHERE line_revenue > 0 GROUP BY country), ranked AS (SELECT *, RANK() OVER (ORDER BY revenue DESC) AS revenue_rank FROM country_metrics) SELECT country, ROUND(revenue,2) AS revenue, customers, revenue_rank FROM ranked WHERE revenue_rank <= 20 AND customers < 50 ORDER BY revenue_rank;
```

Output sample:

country	revenue	customers	revenue_rank
EIRE	3422.48	3	2
Singapore	2622.02	1	3
Netherlands	2153.3	2	4
Japan	2043.48	3	5
Germany	1979.86	48	6

Interpretation: Combines revenue rank with customer count for priority review.

Q16. Which rows have a known customer ID?

Perspective: Customer and Access Analysis | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, customer_id, country, line_revenue FROM online_retail_transactions WHERE customer_id IS NOT NULL LIMIT 20;
```

Output sample:

invoice_no	customer_id	country	line_revenue
536365	17850.0	United Kingdom	15.299999999999999
536381	15311.0	United Kingdom	19.5
536390	17511.0	United Kingdom	68.0
536401	15862.0	United Kingdom	2.5
536407	17850.0	United Kingdom	11.100000000000001

Interpretation: Checks records linked to identified customers.

Q17. Which rows have missing customer ID?

Perspective: Customer and Access Analysis | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, country, description, line_revenue FROM online_retail_transactions WHERE customer_id IS NULL LIMIT 20;
```

Output sample:

invoice_no	country	description	line_revenue
536544	United Kingdom	CHRISTMAS CRAFT LITTLE FRIENDS	16.84
536544	United Kingdom	RED KITCHEN SCALES	33.96
536544	United Kingdom	HANGING JAM JAR T-LIGHT HOLDER	13.52
536544	United Kingdom	LOVE HEART NAPKIN BOX	8.5
536544	United Kingdom	DOORMAT FANCY FONT HOME SWEET HOME	14.43

Interpretation: Shows transactions without a customer identifier.

Q18. Which customers bought from countries outside the UK?

Perspective: Customer and Access Analysis | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, customer_id, country, description FROM online_retail_transactions WHERE country <> 'United Kingdom' AND customer_id IS NOT NULL LIMIT 20;
```

Output sample:

invoice_no	customer_id	country	description
536532	12433.0	Norway	LADIES & GENTLEMEN METAL SIGN
536540	14911.0	EIRE	LETS GO SHOPPING COTTON TOTE BAG
536971	12779.0	Poland	BIG DOUGHNUT FRIDGE MAGNETS
536975	14911.0	EIRE	REGENCY CAKESTAND 3 TIER
537065	12567.0	France	LUNCH BAG RED RETROSPOT

Interpretation: Filters non-UK customer transactions.

Q19. Which customer IDs appear in the data?

Perspective: Customer and Access Analysis | Difficulty: Easy | SQL concept: SELECT

SQL used:

```
SELECT DISTINCT customer_id FROM online_retail_transactions WHERE customer_id IS NOT NULL ORDER BY customer_id LIMIT 50;
```

Output sample:

customer_id
12347.0
12349.0
12350.0
12352.0
12354.0

Interpretation: Lists customer IDs for exploration.

Q20. Which customers have large individual line purchases?

Perspective: Customer and Access Analysis | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, customer_id, country, line_revenue FROM online_retail_transactions WHERE line_revenue > 100 ORDER BY line_revenue DESC LIMIT 20;
```

Output sample:

invoice_no	customer_id	country	line_revenue
550461	15749.0	United Kingdom	3380.0
548813	12744.0	Singapore	2382.92
572869	12798.0	Japan	1718.4
547708	17450.0	United Kingdom	871.56
551191	16698.0	United Kingdom	816.0

Interpretation: Identifies higher-value customer transaction lines.

Q21. Which customers have the highest total spend?

Perspective: Customer and Access Analysis | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT customer_id, ROUND(SUM(line_revenue),2) AS total_spend, COUNT(DISTINCT invoice_no) AS invoices FROM
online_retail_transactions WHERE customer_id IS NOT NULL AND line_revenue > 0 GROUP BY customer_id ORDER BY total_spend DESC
LIMIT 20;
```

Output sample:

customer_id	total_spend	invoices
15749.0	3380.0	1
12744.0	2622.02	4
18102.0	2588.44	7
14646.0	2138.3	20
12798.0	1925.76	2

Interpretation: Aggregates customer spend.

Q22. Which customers bought the most quantity?

Perspective: Customer and Access Analysis | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT customer_id, SUM(quantity) AS total_quantity, COUNT(*) AS lines FROM online_retail_transactions WHERE customer_id IS
NOT NULL AND quantity > 0 GROUP BY customer_id ORDER BY total_quantity DESC LIMIT 20;
```

Output sample:

customer_id	total_quantity	lines
15749.0	2000	1
12798.0	1536	2
14646.0	1422	21
17940.0	1296	1
16684.0	1032	3

Interpretation: Finds high-volume customers.

Q23. Which countries have the highest missing customer ID rate?

Perspective: Customer and Access Analysis | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, COUNT(*) AS total_lines, SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS missing_customer_rows,
ROUND(100.0*SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END)/COUNT(*),2) AS missing_pct FROM online_retail_transactions
GROUP BY country HAVING total_lines >= 20 ORDER BY missing_pct DESC LIMIT 20;
```

Output sample:

country	total_lines	missing_customer_rows	missing_pct
United Kingdom	4588	1253	27.31
EIRE	68	6	8.82
Netherlands	22	0	0.0
Germany	87	0	0.0
France	73	0	0.0

Interpretation: Measures customer data completeness by country.

Q24. What is average customer spend by country?

Perspective: Customer and Access Analysis | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, ROUND(SUM(line_revenue)/NULLIF(COUNT(DISTINCT customer_id),0),2) AS revenue_per_customer, COUNT(DISTINCT
customer_id) AS customers FROM online_retail_transactions WHERE customer_id IS NOT NULL AND line_revenue > 0 GROUP BY country
ORDER BY revenue_per_customer DESC LIMIT 20;
```

Output sample:

country	revenue_per_customer	customers

Singapore	2622.02	1
EIRE	1091.68	3
Netherlands	1076.65	2
Japan	681.16	3
Australia	290.65	3

Interpretation: Compares revenue per identified customer.

Q25. Which customers have many invoices?

Perspective: Customer and Access Analysis | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT customer_id, COUNT(DISTINCT invoice_no) AS invoice_count, ROUND(SUM(line_revenue),2) AS total_spend FROM
online_retail_transactions WHERE customer_id IS NOT NULL GROUP BY customer_id HAVING invoice_count >= 2 ORDER BY invoice_count
DESC, total_spend DESC LIMIT 20;
```

Output sample:

customer_id	invoice_count	total_spend
17841.0	79	444.37
14911.0	48	1786.78
12748.0	41	193.54
14606.0	32	189.12
15311.0	21	360.42

Interpretation: Finds repeat customers.

Q26. Rank customers by total spend.

Perspective: Customer and Access Analysis | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
WITH customer_spend AS (SELECT customer_id, SUM(line_revenue) AS total_spend FROM online_retail_transactions WHERE customer_id
IS NOT NULL AND line_revenue > 0 GROUP BY customer_id) SELECT customer_id, ROUND(total_spend,2) AS total_spend, RANK() OVER
(ORDER BY total_spend DESC) AS spend_rank FROM customer_spend ORDER BY spend_rank LIMIT 20;
```

Output sample:

customer_id	total_spend	spend_rank
15749.0	3380.0	1
12744.0	2622.02	2
18102.0	2588.44	3
14646.0	2138.3	4
12798.0	1925.76	5

Interpretation: Ranks customers by spend.

Q27. Which customers are above the average customer spend?

Perspective: Customer and Access Analysis | Difficulty: Difficult | SQL concept: CTE

SQL used:

```
WITH customer_spend AS (SELECT customer_id, SUM(line_revenue) AS total_spend FROM online_retail_transactions WHERE customer_id
IS NOT NULL AND line_revenue > 0 GROUP BY customer_id), avg_spend AS (SELECT AVG(total_spend) AS avg_total_spend FROM
customer_spend) SELECT customer_id, ROUND(total_spend,2) AS total_spend, ROUND(avg_total_spend,2) AS average_spend FROM
customer_spend, avg_spend WHERE total_spend > avg_total_spend ORDER BY total_spend DESC LIMIT 20;
```

Output sample:

customer_id	total_spend	average_spend
15749.0	3380.0	45.25
12744.0	2622.02	45.25
18102.0	2588.44	45.25
14646.0	2138.3	45.25

12798.0	1925.76	45.25

Interpretation: Uses CTEs to compare customers against average.

Q28. Rank customers within each country by spend.

Perspective: Customer and Access Analysis | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
WITH spend AS (SELECT country, customer_id, SUM(line_revenue) AS total_spend FROM online_retail_transactions WHERE customer_id IS NOT NULL AND line_revenue > 0 GROUP BY country, customer_id) SELECT country, customer_id, ROUND(total_spend,2) AS total_spend, ROW_NUMBER() OVER (PARTITION BY country ORDER BY total_spend DESC) AS country_customer_rank FROM spend ORDER BY country, country_customer_rank LIMIT 50;
```

Output sample:

country	customer_id	total_spend	country_customer_rank
Australia	12415.0	839.95	1
Australia	12388.0	17.0	2
Australia	12393.0	15.0	3
Austria	12360.0	27.8	1
Austria	12453.0	10.5	2

Interpretation: Uses partitioned ranking to find leading customers in each country.

Q29. Use the customer value view to identify top customers.

Perspective: Customer and Access Analysis | Difficulty: Difficult | SQL concept: VIEW

SQL used:

```
SELECT customer_id, ROUND(total_spend,2) AS total_spend, invoice_count, country_count FROM v_customer_value ORDER BY total_spend DESC LIMIT 20;
```

Output sample:

customer_id	total_spend	invoice_count	country_count
15749.0	3380.0	1	1
12744.0	2622.02	4	1
18102.0	2588.44	7	1
14646.0	2138.3	20	1
12798.0	1925.76	2	1

Interpretation: Uses a reusable customer value view.

Q30. Which countries have low customer coverage but high spend?

Perspective: Customer and Access Analysis | Difficulty: Difficult | SQL concept: CTE

SQL used:

```
WITH country_metrics AS (SELECT country, COUNT(DISTINCT customer_id) AS customers, SUM(line_revenue) AS revenue FROM online_retail_transactions WHERE line_revenue > 0 GROUP BY country) SELECT country, customers, ROUND(revenue,2) AS revenue FROM country_metrics WHERE customers < 20 AND revenue > (SELECT AVG(revenue) FROM country_metrics) ORDER BY revenue DESC;
```

Output sample:

country	customers	revenue
EIRE	3	3422.48

Interpretation: Highlights countries with concentrated sales but few customers.

Q31. What are sample products in the data?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Easy | SQL concept: SELECT

SQL used:

```
SELECT stock_code, description, quantity, unit_price FROM online_retail_transactions LIMIT 20;
```

Output sample:

stock_code	description	quantity	unit_price
85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2.55
22411	JUMBO SHOPPER VINTAGE RED PAISLEY	10	1.95
22968	ROSE COTTAGE KEEPSAKE BOX	8	8.5
85049E	SCANDINAVIAN REDS RIBBONS	2	1.25
22632	HAND WARMER RED POLKA DOT	6	1.85

Interpretation: Shows product line examples.

Q32. Which products have negative quantities?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, stock_code, description, quantity FROM online_retail_transactions WHERE quantity < 0 LIMIT 20;
```

Output sample:

invoice_no	stock_code	description	quantity
C536814	85066	CREAM SWEETHEART MINI CHEST	-2
C537602	22065	CHRISTMAS PUDDING TRINKET POT	-1
C538341	22728	ALARM CLOCK BAKELIKE PINK	-3
C538350	82551	LAUNDRY 15C METAL SIGN	-1
C538692	35001W	HAND OPEN SHAPE DECO.WHITE	-1

Interpretation: Shows likely returns or corrections.

Q33. Which products have zero or missing unit price?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, stock_code, description, unit_price FROM online_retail_transactions WHERE unit_price IS NULL OR unit_price
<= 0 LIMIT 20;
```

Output sample:

invoice_no	stock_code	description	unit_price
538140	90002D		0.0
542368	85188B		0.0
542555	21841		0.0
542965	35400		0.0
546933	22521	CHILDS GARDEN TROWEL PINK	0.0

Interpretation: Finds free or invalid price records.

Q34. Which products are high priced?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT stock_code, description, unit_price FROM online_retail_transactions WHERE unit_price > 10 ORDER BY unit_price DESC
LIMIT 20;
```

Output sample:

stock_code	description	unit_price
M	Manual	2382.92
DOT	DOTCOM POSTAGE	449.52

DOT	DOTCOM POSTAGE	264.0
DOT	DOTCOM POSTAGE	247.54
DOT	DOTCOM POSTAGE	176.41

Interpretation: Filters higher-price products.

Q35. Which unique stock codes exist?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Easy | SQL concept: SELECT

SQL used:

```
SELECT DISTINCT stock_code, description FROM online_retail_transactions WHERE stock_code <> '' ORDER BY stock_code LIMIT 50;
```

Output sample:

stock_code	description
10002	INFLATABLE POLITICAL GLOBE
10125	MINI FUNKY DESIGN TAPES
10133	COLOURING PENCILS BROWN TUBE
10135	COLOURING PENCILS BROWN TUBE
11001	ASSTD DESIGN RACING CAR PEN

Interpretation: Lists product codes.

Q36. Which products generate the highest revenue?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT stock_code, description, ROUND(SUM(line_revenue),2) AS total_revenue, SUM(quantity) AS quantity_sold FROM
online_retail_transactions WHERE line_revenue > 0 GROUP BY stock_code, description ORDER BY total_revenue DESC LIMIT 20;
```

Output sample:

stock_code	description	total_revenue	quantity_sold
21175	GIN + TONIC DIET METAL SIGN	3642.9	2111
23084	RABBIT NIGHT LIGHT	2550.45	1371
M	Manual	2404.62	10
DOT	DOTCOM POSTAGE	1260.05	5
22423	REGENCY CAKESTAND 3 TIER	1230.54	71

Interpretation: Identifies strongest revenue products.

Q37. Which products sell the most units?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT stock_code, description, SUM(quantity) AS total_quantity FROM online_retail_transactions WHERE quantity > 0 GROUP BY
stock_code, description ORDER BY total_quantity DESC LIMIT 20;
```

Output sample:

stock_code	description	total_quantity
21175	GIN + TONIC DIET METAL SIGN	2111
21985	PACK OF 12 HEARTS DESIGN TISSUES	1453
23084	RABBIT NIGHT LIGHT	1371
15036	ASSORTED COLOURS SILK FAN	744
23307	SET OF 60 PANTRY DESIGN CAKE CASES	587

Interpretation: Identifies high-volume products.

Q38. Which products have the highest average price?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT stock_code, description, ROUND(AVG(unit_price),2) AS avg_price, COUNT(*) AS lines FROM online_retail_transactions WHERE unit_price > 0 GROUP BY stock_code, description HAVING COUNT(*) >= 2 ORDER BY avg_price DESC LIMIT 20;
```

Output sample:

stock_code	description	avg_price	lines
M	Manual	342.63	7
DOT	DOTCOM POSTAGE	252.01	5
22839	3 TIER CAKE TIN GREEN AND CREAM	22.06	2
23253	16 PIECE CUTLERY SET PANTRY DESIGN	20.52	3
23007	SPACEBOY BABY GIFT SET	16.95	2

Interpretation: Compares average product prices.

Q39. Which products appear in the most countries?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT stock_code, description, COUNT(DISTINCT country) AS country_count FROM online_retail_transactions GROUP BY stock_code, description ORDER BY country_count DESC LIMIT 20;
```

Output sample:

stock_code	description	country_count
POST	POSTAGE	6
22027	TEA PARTY BIRTHDAY CARD	4
22326	ROUND SNACK BOXES SET OF4 WOODLAND	4
22423	REGENCY CAKESTAND 3 TIER	4
22554	PLASTERS IN TIN WOODLAND ANIMALS	4

Interpretation: Finds products with broad geographic reach.

Q40. Which products have frequent negative quantity rows?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT stock_code, description, COUNT(*) AS negative_rows FROM online_retail_transactions WHERE quantity < 0 GROUP BY stock_code, description ORDER BY negative_rows DESC LIMIT 20;
```

Output sample:

stock_code	description	negative_rows
22720	SET OF 3 CAKE TINS PANTRY DESIGN	4
21314	SMALL GLASS HEART TRINKET POT	2
21537	RED RETROSPOT PUDDING BOWL	2
22356	CHARLOTTE BAG PINK POLKADOT	2
22665	RECIPE BOX BLUE SKETCHBOOK DESIGN	2

Interpretation: Highlights products with frequent returns/corrections.

Q41. Rank products by revenue within each country.

Perspective: Product Demand and Catalogue Analysis | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
WITH product_country AS (SELECT country, stock_code, description, SUM(line_revenue) AS revenue FROM online_retail_transactions WHERE line_revenue > 0 GROUP BY country, stock_code, description) SELECT country, stock_code, description, ROUND(revenue,2) AS revenue, ROW_NUMBER() OVER (PARTITION BY country ORDER BY revenue DESC) AS product_rank_in_country FROM product_country ORDER BY country, product_rank_in_country LIMIT 50;
```

Output sample:

country	stock_code	description	revenue	product_rank_in_country
Australia	22617	BAKING SET SPACEBOY DESIGN	306.0	1
Australia	21974	SET OF 36 PAISLEY FLOWER DOILIES	180.0	2
Australia	22749	FELTCRAFT PRINCESS CHARLOTTE DOLL	162.72	3
Australia	22148	EASTER CRAFT 4 CHICKS	66.0	4
Australia	22918	HERB MARKER PARSLEY	66.0	5

Interpretation: Ranks products within each country.

Q42. Which products are above average revenue?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Difficult | SQL concept: CTE

SQL used:

```
WITH product_revenue AS (SELECT stock_code, description, SUM(line_revenue) AS revenue FROM online_retail_transactions WHERE line_revenue > 0 GROUP BY stock_code, description), avg_product AS (SELECT AVG(revenue) AS avg_revenue FROM product_revenue) SELECT stock_code, description, ROUND(revenue,2) AS revenue, ROUND(avg_revenue,2) AS avg_revenue FROM product_revenue, avg_product WHERE revenue > avg_revenue ORDER BY revenue DESC LIMIT 20;
```

Output sample:

stock_code	description	revenue	avg_revenue
21175	GIN + TONIC DIET METAL SIGN	3642.9	51.89
23084	RABBIT NIGHT LIGHT	2550.45	51.89
M	Manual	2404.62	51.89
DOT	DOTCOM POSTAGE	1260.05	51.89
22423	REGENCY CAKESTAND 3 TIER	1230.54	51.89

Interpretation: Compares product revenue to average.

Q43. What is the cumulative revenue contribution by product?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
WITH product_revenue AS (SELECT stock_code, description, SUM(line_revenue) AS revenue FROM online_retail_transactions WHERE line_revenue > 0 GROUP BY stock_code, description) SELECT stock_code, description, ROUND(revenue,2) AS revenue, ROUND(SUM(revenue) OVER (ORDER BY revenue DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW),2) AS cumulative_revenue FROM product_revenue ORDER BY revenue DESC LIMIT 20;
```

Output sample:

stock_code	description	revenue	cumulative_revenue
21175	GIN + TONIC DIET METAL SIGN	3642.9	3642.9
23084	RABBIT NIGHT LIGHT	2550.45	6193.35
M	Manual	2404.62	8597.97
DOT	DOTCOM POSTAGE	1260.05	9858.02
22423	REGENCY CAKESTAND 3 TIER	1230.54	11088.56

Interpretation: Uses running total to show cumulative product contribution.

Q44. Use product performance view to find leading products.

Perspective: Product Demand and Catalogue Analysis | Difficulty: Difficult | SQL concept: VIEW

SQL used:

```
SELECT stock_code, description, ROUND(total_revenue,2) AS total_revenue, total_quantity FROM v_product_performance ORDER BY total_revenue DESC LIMIT 20;
```

Output sample:

stock_code	description	total_revenue	total_quantity
21175	GIN + TONIC DIET METAL SIGN	3642.9	2111
23084	RABBIT NIGHT LIGHT	2550.45	1371
M	Manual	2404.62	10
DOT	DOTCOM POSTAGE	1260.05	5
22423	REGENCY CAKESTAND 3 TIER	1230.54	71

Interpretation: Uses a reusable product performance view.

Q45. Which high-volume products have lower average prices?

Perspective: Product Demand and Catalogue Analysis | Difficulty: Difficult | SQL concept: CTE

SQL used:

```
WITH product_metrics AS (SELECT stock_code, description, SUM(quantity) AS qty, AVG(unit_price) AS avg_price FROM online_retail_transactions WHERE quantity > 0 AND unit_price > 0 GROUP BY stock_code, description) SELECT stock_code, description, qty, ROUND(avg_price,2) AS avg_price FROM product_metrics WHERE qty > (SELECT AVG(qty) FROM product_metrics) AND avg_price < (SELECT AVG(avg_price) FROM product_metrics) ORDER BY qty DESC LIMIT 20;
```

Output sample:

stock_code	description	qty	avg_price
21175	GIN + TONIC DIET METAL SIGN	2111	2.69
21985	PACK OF 12 HEARTS DESIGN TISSUES	1453	0.41
23084	RABBIT NIGHT LIGHT	1371	2.29
15036	ASSORTED COLOURS SILK FAN	744	0.75
23307	SET OF 60 PANTRY DESIGN CAKE CASES	587	0.73

Interpretation: Finds products with strong volume and lower price.

Q46. Which invoices are cancellations?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, stock_code, description, quantity, line_revenue FROM online_retail_transactions WHERE is_cancelled = 1 LIMIT 20;
```

Output sample:

invoice_no	stock_code	description	quantity	line_revenue
C536814	85066	CREAM SWEETHEART MINI CHEST	-2	-25.5
C537602	22065	CHRISTMAS PUDDING TRINKET POT	-1	-1.25
C538341	22728	ALARM CLOCK BAKELIKE PINK	-3	-11.25
C538350	82551	LAUNDRY 15C METAL SIGN	-1	-1.25
C538692	35001W	HAND OPEN SHAPE DECO.WHITE	-1	-3.49

Interpretation: Shows cancelled invoice rows.

Q47. Which rows have negative revenue?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, stock_code, description, line_revenue FROM online_retail_transactions WHERE line_revenue < 0 ORDER BY line_revenue LIMIT 20;
```

Output sample:

invoice_no	stock_code	description	line_revenue
C570647	23343	JUMBO BAG VINTAGE CHRISTMAS	-179.0
C555006	23300	GARDENERS KNEELING PAD CUP OF TEA	-139.2
C548577	21873	IF YOU CAN'T STAND THE HEAT MUG	-76.32000000000001
C558405	22720	SET OF 3 CAKE TINS PANTRY DESIGN	-68.0
C560869	D	Discount	-66.24

Interpretation: Finds returns or credit lines.

Q48. Which rows are missing descriptions?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, stock_code, description FROM online_retail_transactions WHERE description IS NULL OR description = '' LIMIT 20;
```

Output sample:

invoice_no	stock_code	description
538140	90002D	
542368	85188B	
542555	21841	
542965	35400	
547529	21841	

Interpretation: Checks product description completeness.

Q49. Which rows are missing country?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, stock_code, country FROM online_retail_transactions WHERE country IS NULL OR country = '' LIMIT 20;
```

Output sample:

invoice_no	stock_code	country

Interpretation: Checks country completeness.

Q50. Which rows have invalid quantity or price?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, stock_code, quantity, unit_price FROM online_retail_transactions WHERE quantity IS NULL OR unit_price IS NULL LIMIT 20;
```

Output sample:

invoice_no	stock_code	quantity	unit_price

Interpretation: Finds records with missing numeric fields.

Q51. Which countries have the most cancellations?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, COUNT(*) AS cancelled_lines FROM online_retail_transactions WHERE is_cancelled = 1 GROUP BY country ORDER BY cancelled_lines DESC LIMIT 20;
```

Output sample:

country	cancelled_lines
United Kingdom	68
Germany	4
USA	2
Switzerland	2
France	2

Interpretation: Counts cancelled rows by country.

Q52. What percentage of rows are cancelled by country?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, COUNT(*) AS total_lines, SUM(is_cancelled) AS cancelled_lines, ROUND(100.0*SUM(is_cancelled)/COUNT(*),2) AS cancellation_pct FROM online_retail_transactions GROUP BY country HAVING total_lines >= 20 ORDER BY cancellation_pct DESC LIMIT 20;
```

Output sample:

country	total_lines	cancelled_lines	cancellation_pct
Germany	87	4	4.6
EIRE	68	2	2.94
France	73	2	2.74
United Kingdom	4588	68	1.48
Netherlands	22	0	0.0

Interpretation: Compares cancellation rates.

Q53. Which products have the largest negative revenue?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT stock_code, description, ROUND(SUM(line_revenue),2) AS net_negative_revenue FROM online_retail_transactions WHERE line_revenue < 0 GROUP BY stock_code, description ORDER BY net_negative_revenue ASC LIMIT 20;
```

Output sample:

stock_code	description	net_negative_revenue
23343	JUMBO BAG VINTAGE CHRISTMAS	-179.0
23300	GARDENERS KNEELING PAD CUP OF TEA	-139.2
22720	SET OF 3 CAKE TINS PANTRY DESIGN	-87.8
21873	IF YOU CAN'T STAND THE HEAT MUG	-76.32
D	Discount	-66.24

Interpretation: Finds products with largest negative values.

Q54. Which countries have missing customer IDs?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, COUNT(*) AS rows_count, SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS missing_customer_rows FROM online_retail_transactions GROUP BY country ORDER BY missing_customer_rows DESC LIMIT 20;
```

Output sample:

country	rows_count	missing_customer_rows
United Kingdom	4588	1253
EIRE	68	6
Unspecified	3	2
Hong Kong	2	2

Portugal	19	1

Interpretation: Checks customer ID missingness by country.

Q55. Which product descriptions appear against multiple stock codes?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT description, COUNT(DISTINCT stock_code) AS stock_code_count, COUNT(*) AS lines FROM online_retail_transactions WHERE
description <> '' GROUP BY description HAVING stock_code_count > 1 ORDER BY stock_code_count DESC LIMIT 20;
```

Output sample:

description	stock_code_count	lines
damaged	2	2
check	2	2
WHITE HANGING HEART T-LIGHT HOLDER	2	28
TRADITIONAL CHRISTMAS RIBBONS	2	6
TEA TIME PARTY BUNTING	2	3

Interpretation: Detects catalogue consistency issues.

Q56. Which countries have cancellation rates above average?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Difficult | SQL concept: CTE

SQL used:

```
WITH country_cancel AS (SELECT country, COUNT(*) AS total_lines, SUM(is_cancelled) AS cancelled_lines,
1.0*SUM(is_cancelled)/COUNT(*) AS cancel_rate FROM online_retail_transactions GROUP BY country HAVING total_lines >= 20),
avg_cancel AS (SELECT AVG(cancel_rate) AS avg_rate FROM country_cancel) SELECT country, total_lines, cancelled_lines,
ROUND(100*cancel_rate,2) AS cancel_pct, ROUND(100*avg_rate,2) AS avg_cancel_pct FROM country_cancel, avg_cancel WHERE
cancel_rate > avg_rate ORDER BY cancel_rate DESC;
```

Output sample:

country	total_lines	cancelled_lines	cancel_pct	avg_cancel_pct
Germany	87	4	4.6	2.35
EIRE	68	2	2.94	2.35
France	73	2	2.74	2.35

Interpretation: Compares country cancellation rates to average.

Q57. Rank products by number of cancellation rows.

Perspective: Returns, Cancellations and Data Quality | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
WITH product_cancel AS (SELECT stock_code, description, COUNT(*) AS cancelled_rows FROM online_retail_transactions WHERE
is_cancelled = 1 GROUP BY stock_code, description) SELECT stock_code, description, cancelled_rows, RANK() OVER (ORDER BY
cancelled_rows DESC) AS cancellation_rank FROM product_cancel ORDER BY cancellation_rank LIMIT 20;
```

Output sample:

stock_code	description	cancelled_rows	cancellation_rank
22720	SET OF 3 CAKE TINS PANTRY DESIGN	4	1
21314	SMALL GLASS HEART TRINKET POT	2	2
21537	RED RETROSPOT PUDDING BOWL	2	2
22356	CHARLOTTE BAG PINK POLKADOT	2	2
22665	RECIPE BOX BLUE SKETCHBOOK DESIGN	2	2

Interpretation: Ranks products by cancellation frequency.

Q58. Show cancellation trend by month and previous month gap.

Perspective: Returns, Cancellations and Data Quality | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
WITH monthly AS (SELECT invoice_month, COUNT(*) AS total_lines, SUM(is_cancelled) AS cancelled_lines FROM
online_retail_transactions WHERE invoice_month IS NOT NULL GROUP BY invoice_month) SELECT invoice_month, total_lines,
cancelled_lines, cancelled_lines - LAG(cancelled_lines) OVER (ORDER BY invoice_month) AS change_from_previous_month FROM
monthly ORDER BY invoice_month LIMIT 20;
```

Output sample:

invoice_month	total_lines	cancelled_lines	change_from_previous_month
2010-12	392	6	None
2011-01	325	5	-1
2011-02	255	4	-1
2011-03	339	4	0
2011-04	276	7	3

Interpretation: Uses LAG to compare cancellation counts month to month.

Q59. Use the data quality view to show country completeness.

Perspective: Returns, Cancellations and Data Quality | Difficulty: Difficult | SQL concept: VIEW

SQL used:

```
SELECT country, total_lines, missing_customer_rows, ROUND(missing_customer_pct,2) AS missing_customer_pct FROM
v_country_data_quality ORDER BY missing_customer_pct DESC LIMIT 20;
```

Output sample:

country	total_lines	missing_customer_rows	missing_customer_pct
Hong Kong	2	2	100.0
Unspecified	3	2	66.67
United Kingdom	4588	1253	27.31
EIRE	68	6	8.82
Portugal	19	1	5.26

Interpretation: Uses a reusable data-quality view.

Q60. Which countries combine high revenue and high cancellation percentage?

Perspective: Returns, Cancellations and Data Quality | Difficulty: Difficult | SQL concept: CTE

SQL used:

```
WITH country_metrics AS (SELECT country, SUM(line_revenue) AS revenue, COUNT(*) AS total_lines, SUM(is_cancelled) AS
cancelled_lines, 1.0*SUM(is_cancelled)/COUNT(*) AS cancel_rate FROM online_retail_transactions GROUP BY country HAVING
total_lines >= 20) SELECT country, ROUND(revenue,2) AS revenue, total_lines, ROUND(100*cancel_rate,2) AS cancel_pct FROM
country_metrics WHERE revenue > (SELECT AVG(revenue) FROM country_metrics) AND cancel_rate > (SELECT AVG(cancel_rate) FROM
country_metrics) ORDER BY revenue DESC;
```

Output sample:

country	revenue	total_lines	cancel_pct

Interpretation: Finds markets that are important but potentially risky.

Q61. Which transactions happened in the first invoice month?

Perspective: Time Trends and Operational Planning | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, invoice_date_clean, invoice_month, country, line_revenue FROM online_retail_transactions WHERE
invoice_month = (SELECT MIN(invoice_month) FROM online_retail_transactions WHERE invoice_month IS NOT NULL) LIMIT 20;
```

Output sample:

invoice_no	invoice_date_clean	invoice_month	country	line_revenue
536365	2010-12-01 08:26:00	2010-12	United Kingdom	15.299999999999999
536381	2010-12-01 09:41:00	2010-12	United Kingdom	19.5
536390	2010-12-01 10:19:00	2010-12	United Kingdom	68.0
536401	2010-12-01 11:21:00	2010-12	United Kingdom	2.5
536407	2010-12-01 11:34:00	2010-12	United Kingdom	11.100000000000001

Interpretation: Shows data from the earliest month.

Q62. Which months are present in the dataset?

Perspective: Time Trends and Operational Planning | Difficulty: Easy | SQL concept: SELECT

SQL used:

```
SELECT DISTINCT invoice_month FROM online_retail_transactions WHERE invoice_month IS NOT NULL ORDER BY invoice_month LIMIT 50;
```

Output sample:

invoice_month
2010-12
2011-01
2011-02
2011-03
2011-04

Interpretation: Lists months available.

Q63. Which transactions happened during business hours?

Perspective: Time Trends and Operational Planning | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, invoice_hour, description, line_revenue FROM online_retail_transactions WHERE invoice_hour BETWEEN 9 AND 17
LIMIT 20;
```

Output sample:

invoice_no	invoice_hour	description	line_revenue
536381	9	JUMBO SHOPPER VINTAGE RED PAISLEY	19.5
536390	10	ROSE COTTAGE KEEPSAKE BOX	68.0
536401	11	SCANDINAVIAN REDS RIBBONS	2.5
536407	11	HAND WARMER RED POLKA DOT	11.100000000000001
536412	11	LUNCH BAG SPACEBOY DESIGN	4.949999999999999

Interpretation: Filters transactions by time of day.

Q64. Which transactions happened outside business hours?

Perspective: Time Trends and Operational Planning | Difficulty: Easy | SQL concept: WHERE

SQL used:

```
SELECT invoice_no, invoice_hour, description, line_revenue FROM online_retail_transactions WHERE invoice_hour < 9 OR
invoice_hour > 17 LIMIT 20;
```

Output sample:

invoice_no	invoice_hour	description	line_revenue
536365	8	WHITE HANGING HEART T-LIGHT HOLDER	15.299999999999999
536602	8	WHITE METAL LANTERN	20.34
536841	18	ANTIQUE GLASS HEART DECORATION	23.4
536846	19	TRADITIONAL WOODEN SKIPPING ROPE	2.5

537666	18	RECYCLING BAG RETROSPOT	16.84

Interpretation: Shows transactions outside common working hours.

Q65. Which weekdays appear in the dataset?

Perspective: Time Trends and Operational Planning | Difficulty: Easy | SQL concept: SELECT

SQL used:

```
SELECT DISTINCT invoice_weekday FROM online_retail_transactions WHERE invoice_weekday <> '' ORDER BY invoice_weekday;
```

Output sample:

invoice_weekday
Friday
Monday
Sunday
Thursday
Tuesday

Interpretation: Lists transaction weekdays.

Q66. What is monthly revenue?

Perspective: Time Trends and Operational Planning | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT invoice_month, ROUND(SUM(line_revenue),2) AS monthly_revenue, COUNT(*) AS lines FROM online_retail_transactions WHERE
invoice_month IS NOT NULL GROUP BY invoice_month ORDER BY invoice_month;
```

Output sample:

invoice_month	monthly_revenue	lines
2010-12	5240.31	392
2011-01	6233.81	325
2011-02	4990.52	255
2011-03	6730.48	339
2011-04	11705.97	276

Interpretation: Aggregates revenue by month.

Q67. Which weekdays have the most transaction lines?

Perspective: Time Trends and Operational Planning | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT invoice_weekday, COUNT(*) AS transaction_lines, ROUND(SUM(line_revenue),2) AS total_revenue FROM
online_retail_transactions WHERE invoice_weekday <> '' GROUP BY invoice_weekday ORDER BY transaction_lines DESC;
```

Output sample:

invoice_weekday	transaction_lines	total_revenue
Thursday	955	19589.98
Tuesday	935	18063.83
Wednesday	879	19219.48
Monday	879	20196.84
Friday	758	12588.34

Interpretation: Shows activity by weekday.

Q68. Which hours generate the most revenue?

Perspective: Time Trends and Operational Planning | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT invoice_hour, ROUND(SUM(line_revenue),2) AS revenue, COUNT(*) AS lines FROM online_retail_transactions WHERE
invoice_hour IS NOT NULL GROUP BY invoice_hour ORDER BY revenue DESC LIMIT 24;
```

Output sample:

invoice_hour	revenue	lines
13	17449.19	655
11	12507.0	543
12	12008.19	734
10	11714.86	448
14	11272.3	625

Interpretation: Aggregates revenue by hour.

Q69. How does monthly cancellation count change?

Perspective: Time Trends and Operational Planning | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT invoice_month, SUM(is_cancelled) AS cancellations, COUNT(*) AS total_lines FROM online_retail_transactions WHERE
invoice_month IS NOT NULL GROUP BY invoice_month ORDER BY invoice_month;
```

Output sample:

invoice_month	cancellations	total_lines
2010-12	6	392
2011-01	5	325
2011-02	4	255
2011-03	4	339
2011-04	7	276

Interpretation: Shows monthly cancellations.

Q70. Which country-month combinations generate highest revenue?

Perspective: Time Trends and Operational Planning | Difficulty: Medium | SQL concept: GROUP BY

SQL used:

```
SELECT country, invoice_month, ROUND(SUM(line_revenue),2) AS revenue FROM online_retail_transactions WHERE invoice_month IS
NOT NULL GROUP BY country, invoice_month ORDER BY revenue DESC LIMIT 20;
```

Output sample:

country	invoice_month	revenue
United Kingdom	2011-11	10776.61
United Kingdom	2011-04	9033.7
United Kingdom	2011-10	8453.66
United Kingdom	2011-09	7583.87
United Kingdom	2011-03	5539.35

Interpretation: Finds high-revenue country-month combinations.

Q71. Show monthly revenue with previous month comparison.

Perspective: Time Trends and Operational Planning | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
WITH monthly AS (SELECT invoice_month, SUM(line_revenue) AS revenue FROM online_retail_transactions WHERE invoice_month IS NOT
NULL GROUP BY invoice_month) SELECT invoice_month, ROUND(revenue,2) AS revenue, ROUND(LAG(revenue) OVER (ORDER BY
invoice_month),2) AS previous_month_revenue, ROUND(revenue - LAG(revenue) OVER (ORDER BY invoice_month),2) AS monthly_change
FROM monthly ORDER BY invoice_month;
```

Output sample:

invoice_month	revenue	previous_month_revenue	monthly_change
2010-12	5240.31	None	None

2011-01	6233.81	5240.31	993.5
2011-02	4990.52	6233.81	-1243.29
2011-03	6730.48	4990.52	1739.96
2011-04	11705.97	6730.48	4975.49

Interpretation: Uses LAG to show month-on-month change.

Q72. Calculate running monthly revenue.

Perspective: Time Trends and Operational Planning | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
WITH monthly AS (SELECT invoice_month, SUM(line_revenue) AS revenue FROM online_retail_transactions WHERE invoice_month IS NOT NULL GROUP BY invoice_month) SELECT invoice_month, ROUND(revenue,2) AS revenue, ROUND(SUM(revenue) OVER (ORDER BY invoice_month ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW),2) AS running_revenue FROM monthly ORDER BY invoice_month;
```

Output sample:

invoice_month	revenue	running_revenue
2010-12	5240.31	5240.31
2011-01	6233.81	11474.12
2011-02	4990.52	16464.64
2011-03	6730.48	23195.12
2011-04	11705.97	34901.09

Interpretation: Uses a window running total.

Q73. Which months are above average revenue?

Perspective: Time Trends and Operational Planning | Difficulty: Difficult | SQL concept: CTE

SQL used:

```
WITH monthly AS (SELECT invoice_month, SUM(line_revenue) AS revenue FROM online_retail_transactions WHERE invoice_month IS NOT NULL GROUP BY invoice_month), avg_month AS (SELECT AVG(revenue) AS avg_revenue FROM monthly) SELECT invoice_month, ROUND(revenue,2) AS revenue, ROUND(avg_revenue,2) AS avg_revenue FROM monthly, avg_month WHERE revenue > avg_revenue ORDER BY revenue DESC;
```

Output sample:

invoice_month	revenue	avg_revenue
2011-11	11902.24	7481.61
2011-04	11705.97	7481.61
2011-10	11381.86	7481.61
2011-09	9934.27	7481.61

Interpretation: Uses CTEs to compare each month to the average.

Q74. Use monthly trend view to show revenue trend.

Perspective: Time Trends and Operational Planning | Difficulty: Difficult | SQL concept: VIEW

SQL used:

```
SELECT invoice_month, ROUND(monthly_revenue,2) AS monthly_revenue, total_lines FROM v_monthly_trends ORDER BY invoice_month;
```

Output sample:

invoice_month	monthly_revenue	total_lines
2010-12	5240.31	392
2011-01	6233.81	325
2011-02	4990.52	255
2011-03	6730.48	339
2011-04	11705.97	276

Interpretation: Uses a reusable monthly trend view.

Q75. Rank months by revenue and show cumulative share.

Perspective: Time Trends and Operational Planning | Difficulty: Difficult | SQL concept: Window Function

SQL used:

```
WITH monthly AS (SELECT invoice_month, SUM(line_revenue) AS revenue FROM online_retail_transactions WHERE invoice_month IS NOT NULL GROUP BY invoice_month), total AS (SELECT SUM(revenue) AS total_revenue FROM monthly) SELECT invoice_month, ROUND(revenue,2) AS revenue, RANK() OVER (ORDER BY revenue DESC) AS revenue_rank, ROUND(100.0*SUM(revenue) OVER (ORDER BY revenue DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)/(SELECT total_revenue FROM total),2) AS cumulative_revenue_pct FROM monthly ORDER BY revenue_rank;
```

Output sample:

invoice_month	revenue	revenue_rank	cumulative_revenue_pct
2011-11	11902.24	1	12.24
2011-04	11705.97	2	24.27
2011-10	11381.86	3	35.98
2011-09	9934.27	4	46.19
2011-03	6730.48	5	53.11

Interpretation: Ranks months and calculates cumulative revenue share.

5. Closing Reflection

The biggest challenge was turning a large operational dataset into structured analytical questions. The most useful insight is that SQL can quickly highlight where revenue, product demand, customer access, returns, and data-quality risks are concentrated. The project uses clear comments and query-by-query explanations so the logic can be explained in the 1-to-1 review call.